

11.6 Red-Black Trees

Although AVL trees and (2,4) trees have a number of nice properties, they also have some disadvantages. For instance, AVL trees may require many restructure operations (rotations) to be performed after a deletion, and (2,4) trees may require many split or fusing operations to be performed after an insertion or removal. The data structure we discuss in this section, the red-black tree, does not have these drawbacks; it uses $O(1)$ structural changes after an update in order to stay balanced.

Formally, a **red-black tree** is a binary search tree (see Section 11.1) with nodes colored red and black in a way that satisfies the following properties:

Root Property: The root is black.

External Property: Every external node is black.

Red Property: The children of a red node are black.

Depth Property: All external nodes have the same **black depth**, defined as the number of *proper* ancestors that are black.

An example of a red-black tree is shown in Figure 11.29.

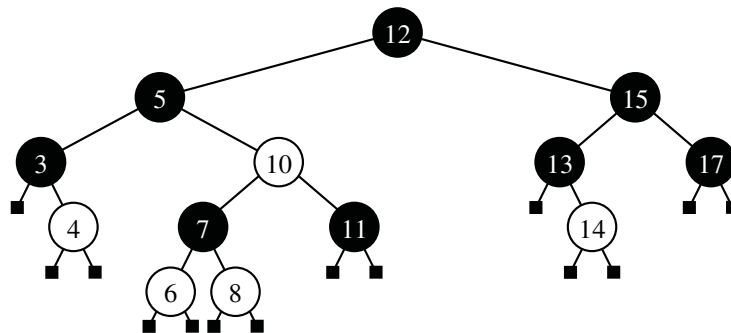


Figure 11.29: An example of a red-black tree, with “red” nodes drawn in white. The common black depth for this tree is 3.

We can make the red-black tree definition more intuitive by noting an interesting correspondence between red-black trees and (2,4) trees. Namely, given a red-black tree, we can construct a corresponding (2,4) tree by merging every red node w into its parent, storing the entry from w at its parent, and with the children of w becoming ordered children of the parent. For example, the red-black tree in Figure 11.29 corresponds to the (2,4) tree from Figure 11.23, as illustrated in Figure 11.30. The depth property of the red-black tree corresponds to the depth property of the (2,4) tree since exactly one black node of the red-black tree contributes to each node of the corresponding (2,4) tree.

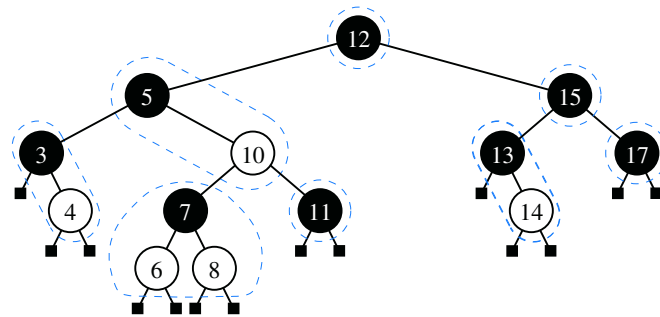


Figure 11.30: An illustration of the correspondence between the red-black tree of Figure 11.29 and the $(2,4)$ tree of Figure 11.23, based on the highlighted grouping of red nodes with their black parents.

Conversely, we can transform any $(2,4)$ tree into a corresponding red-black tree by coloring each node w black and then performing the following transformations, as illustrated in Figure 11.31.

- If w is a 2-node, then keep the (black) children of w as is.
- If w is a 3-node, then create a new red node y , give w 's last two (black) children to y , and make the first child of w and y be the two children of w .
- If w is a 4-node, then create two new red nodes y and z , give w 's first two (black) children to y , give w 's last two (black) children to z , and make y and z be the two children of w .

Notice that a red node always has a black parent in this construction.

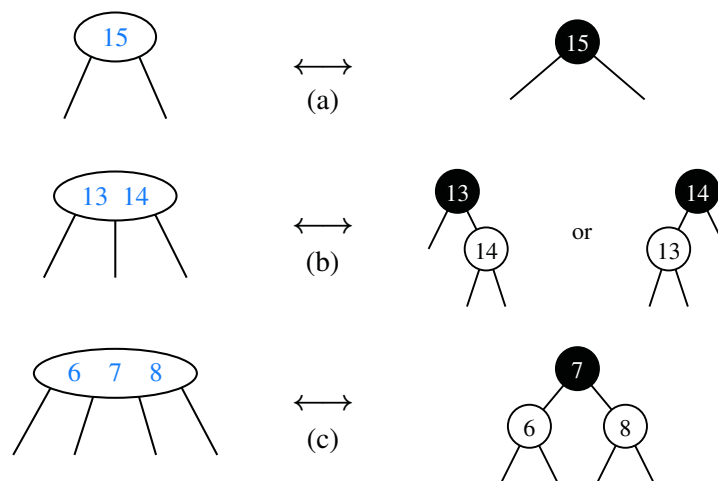


Figure 11.31: Correspondence between nodes of a $(2,4)$ tree and a red-black tree: (a) 2-node; (b) 3-node; (c) 4-node.

Proposition 11.8: *The height of a red-black tree storing n entries is $O(\log n)$.*

Justification: Let T be a red-black tree storing n entries, and let h be the height of T . We justify this proposition by establishing the following fact:

$$\log(n+1) \leq h \leq 2\log(n+1).$$

Let d be the common black depth of all the external nodes of T . Let T' be the $(2,4)$ tree associated with T , and let h' be the height of T' . Because of the correspondence between red-black trees and $(2,4)$ trees, we know that $h' = d$. Hence, by Proposition 11.7, $d = h' \leq \log(n+1)$. By the red property, $h \leq 2d$. Thus, we obtain $h \leq 2\log(n+1)$. The other inequality, $\log(n+1) \leq h$, follows from Proposition 8.7 and the fact that T has n internal nodes. ■

11.6.1 Red-Black Tree Operations

The algorithm for searching in a red-black tree T is the same as that for a standard binary search tree (Section 11.1). Thus, searching in a red-black tree takes time proportional to the height of the tree, which is $O(\log n)$ by Proposition 11.8.

The correspondence between $(2,4)$ trees and red-black trees provides important intuition that we will use in our discussion of how to perform updates in red-black trees; in fact, the update algorithms for red-black trees can seem mysteriously complex without this intuition. Split and fuse operations of a $(2,4)$ tree will be effectively mimicked by recoloring neighboring red-black tree nodes. A rotation within a red-black tree will be used to change orientations of a 3-node between the two forms shown in Figure 11.31(b).

Insertion

Consider the insertion of a key-value pair (k, v) into a red-black tree T . The algorithm initially proceeds as in a standard binary search tree (Section 11.1.2). Namely, we search for k in T and if we reach an external node, we replace this node with an internal node x , storing the entry and having two external children. If this is the first entry in T , and thus x is the root, we color it black. In all other cases, we color x red. That action corresponds to inserting (k, v) into a node of the $(2,4)$ tree T' at the lowest internal level. The insertion preserves the root and depth properties of T , but it may violate the red property. Indeed, if x is not the root of T and its parent y is red, then we have a parent and a child (namely, y and x) that are both red. Note that by the root property, y cannot be the root of T , and by the red property (which was previously satisfied), the parent z of y must be black. Since x and its parent are red, but x 's grandparent z is black, we call this violation of the red property a **double red** at node x . To remedy a double red, we consider two cases.

Case 1: The Sibling s of y is Black. (See Figure 11.32.) In this case, the double red denotes the fact that we have added the new node to a corresponding 3-node of the $(2,4)$ tree T' , effectively creating a malformed 4-node. This formation has one red node, y , that is the parent of another red node, x ; we want the two red nodes to be siblings instead. To fix this problem, we perform a **trinode restructuring** of T . The trinode restructuring (introduced in Section 11.2) is done by the operation $\text{restructure}(x)$, which consists of the following steps (see again Figure 11.32):

- Take node x , its parent y , and grandparent z , and temporarily relabel them as a , b , and c , in left-to-right order, so that a , b , and c will be visited in this order by an inorder tree traversal.
- Replace the grandparent z with the node labeled b , and make nodes a and c the children of b , keeping inorder relationships unchanged.

After performing the $\text{restructure}(x)$ operation, we color b black and we color a and c red. Thus, the restructuring eliminates the double-red problem. Notice that the portion of any path through the restructured part of the tree is incident to exactly one black node, both before and after the trinode restructuring. Therefore, the black depth of the tree is unaffected.

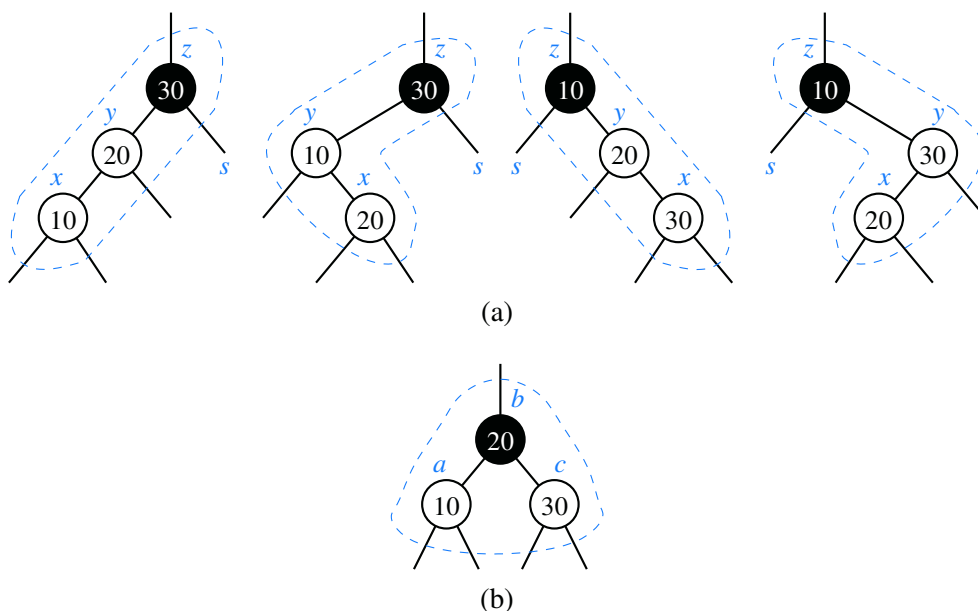


Figure 11.32: Restructuring a red-black tree to remedy a double red: (a) the four configurations for x , y , and z before restructuring; (b) after restructuring.

Case 2: The Siblings of y is Red. (See Figure 11.33.) In this case, the double red denotes an overflow in the corresponding $(2,4)$ tree T' . To fix the problem, we perform the equivalent of a split operation. Namely, we do a **recoloring**: we color y and s black and their parent z red (unless z is the root, in which case, it remains black). Notice that unless z is the root, the portion of any path through the affected part of the tree is incident to exactly one black node, both before and after the recoloring. Therefore, the black depth of the tree is unaffected by the recoloring unless z is the root, in which case it is increased by one.

However, it is possible that the double-red problem reappears after such a recoloring, albeit higher up in the tree T , since z may have a red parent. If the double-red problem reappears at z , then we repeat the consideration of the two cases at z . Thus, a recoloring either eliminates the double-red problem at node x , or propagates it to the grandparent z of x . We continue going up T performing recolorings until we finally resolve the double-red problem (with either a final recoloring or a trinode restructuring). Thus, the number of recolorings caused by an insertion is no more than half the height of tree T , that is, $O(\log n)$ by Proposition 11.8.

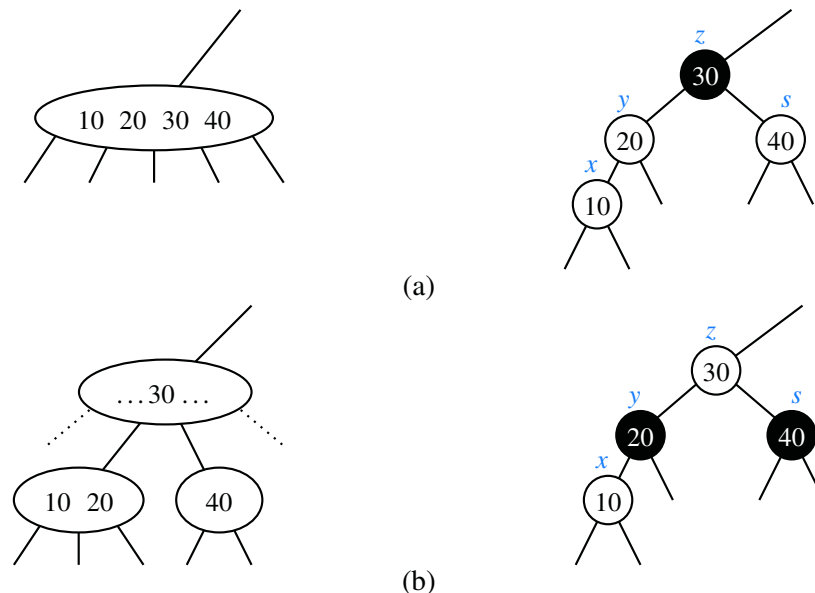


Figure 11.33: Recoloring to remedy the double-red problem: (a) before recoloring and the corresponding 5-node in the associated $(2,4)$ tree before the split; (b) after recoloring and the corresponding nodes in the associated $(2,4)$ tree after the split.

As further examples, Figures 11.34 and 11.35 show a sequence of insertion operations in a red-black tree.

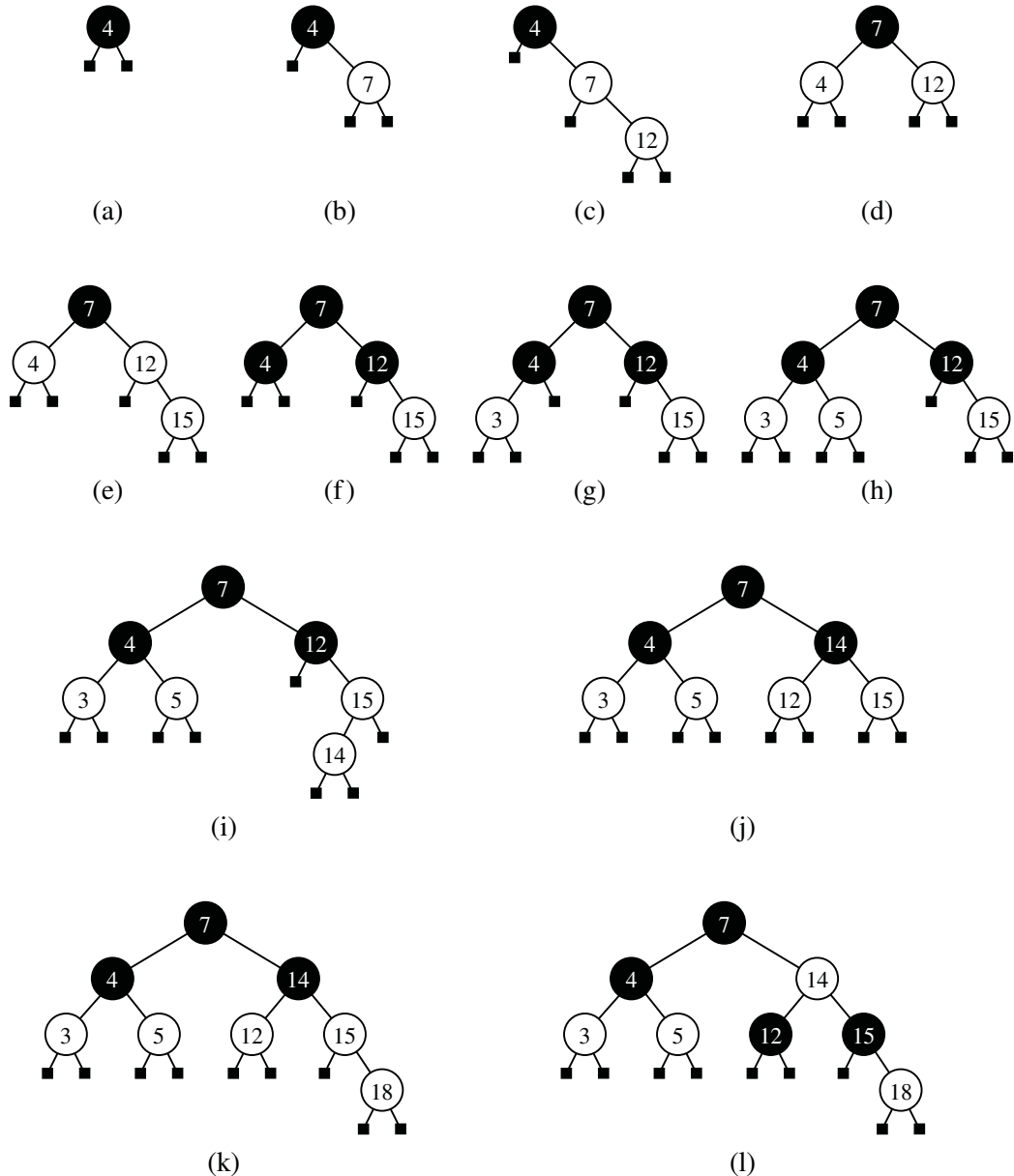


Figure 11.34: A sequence of insertions in a red-black tree: (a) initial tree; (b) insertion of 7; (c) insertion of 12, which causes a double red; (d) after restructuring; (e) insertion of 15, which causes a double red; (f) after recoloring (the root remains black); (g) insertion of 3; (h) insertion of 5; (i) insertion of 14, which causes a double red; (j) after restructuring; (k) insertion of 18, which causes a double red; (l) after recoloring. (Continues in Figure 11.35.)

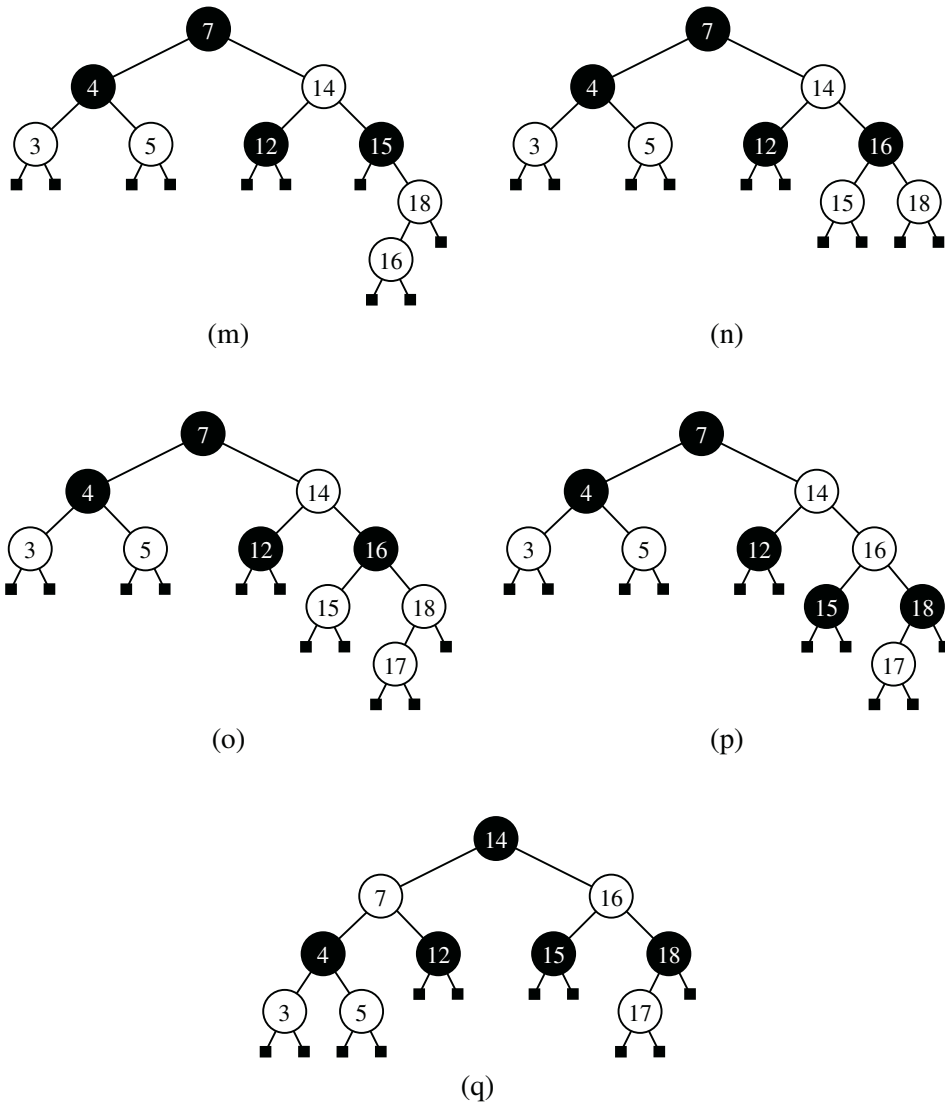


Figure 11.35: A sequence of insertions in a red-black tree (continued from Figure 11.34): (m) insertion of 16, which causes a double red; (n) after restructuring; (o) insertion of 17, which causes a double red; (p) after recoloring there is again a double red, to be handled by a restructuring; (q) after restructuring.

Deletion

Deleting an entry with key k from a red-black tree T initially proceeds as for a binary search tree (Section 11.1.2). Structurally, the process results in the removal of an internal node (either that originally containing key k or its inorder predecessor) together with a child that is external, and the promotion of its other child.

If the removed internal node was red, this structural change does not affect the black depths of any paths in the tree, nor introduce any red violations, and so the resulting tree remains a valid red-black tree. In the corresponding $(2,4)$ tree T' , this case denotes the shrinking of a 4-node or 3-node. If the removed internal node was black, it must have had black height 1, and therefore either both of its children were external, or it had one red child that was an internal node with two external children. In the latter case, the removed node represents the black part of a corresponding 3-node, and we restore the red-black properties by recoloring the promoted child to be black.

The most complex case is when the removed node was black and had two external children. In the corresponding $(2,4)$ tree, this denotes the removal of an entry from a 2-node. Without rebalancing, such a change results in a deficit of one for the black depth of the external position p that is the promoted child of the deleted internal node. To preserve the depth property, we temporarily assign the promoted leaf a fictitious **double black** color. A double black in T denotes an underflow in the corresponding $(2,4)$ tree T' . To remedy a double-black problem at an arbitrary position p , we will consider three cases.

Case 1: The Sibling y of p is Black and has a Red Child x . (See Figure 11.36.)

We perform a **trinode restructuring**, as originally described in Section 11.2. The operation $\text{restructure}(x)$ takes the node x , its parent y , and grandparent z , labels them temporarily left to right as a , b , and c , and replaces z with the node labeled b , making it the parent of the other two. We color a and c black, and give b the former color of z .

Notice that the path to p in the result includes one additional black node after the restructure, while the number of black nodes on paths to any of the other three subtrees illustrated in Figure 11.36 remains unchanged. Therefore, we return p to be colored (regular) black, and the double-black problem is eliminated.

Resolving this case corresponds to a transfer operation in the $(2,4)$ tree T' between two children of node z . The fact that y has a red child assures us that it represents either a 3-node or a 4-node. In effect, the entry previously stored at z is demoted to become a new 2-node to resolve the deficiency, while an entry stored at y or its child is promoted to take the place of the entry previously stored at z .

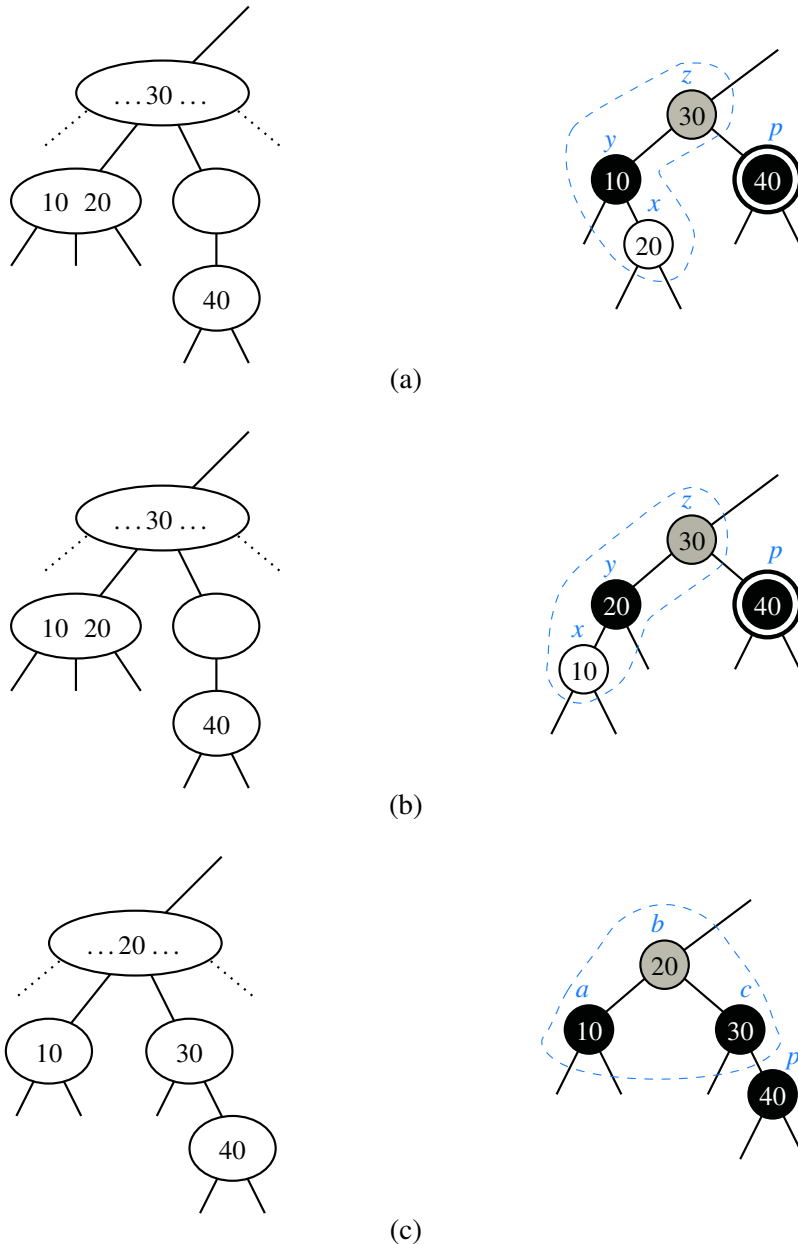


Figure 11.36: Restructuring of a red-black tree to remedy the double-black problem: (a) and (b) configurations before the restructuring, where p is a right child and the associated nodes in the corresponding $(2,4)$ tree before the transfer (two other symmetric configurations where p is a left child are possible); (c) configuration after the restructuring and the associated nodes in the corresponding $(2,4)$ tree after the transfer. The gray color for node z in parts (a) and (b) and for node b in part (c) denotes the fact that this node may be colored either red or black.

Case 2: The Sibling y of p is Black and Both Children of y are Black.

We do a **recoloring**, beginning by changing the color of p from double black to black and the color of y from black to red. This does not create any red violation, because both children of y are black. To counteract the decrease in black depth for paths passing through y or p , we consider the common parent of p and y , which we denote as z . If z is red, we color it black and the problem has been resolved (see Figure 11.37a). If z is black, we color it **double black**, thereby propagating the problem higher up the tree (see Figure 11.37b).

Resolving this case corresponds to a fusion operation in the corresponding $(2,4)$ tree T' , as y must represent a 2-node. The case where the problem propagates upward is when parent z also represents a 2-node.

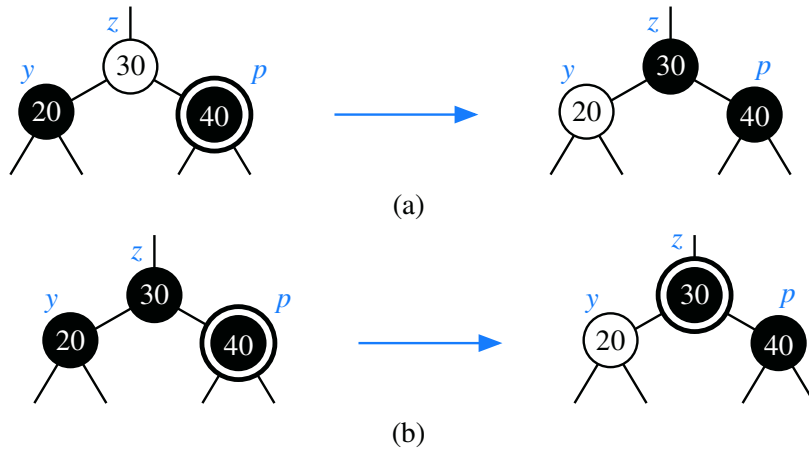


Figure 11.37: A recoloring operation, which has neutral effect on the black depth for paths: (a) when z is originally red, the recoloring resolves the double-black problem, ending the process; (b) when z is originally black, it becomes double-black, requiring a cascading remedy.

Case 3: Sibling y of p is Red. (See Figure 11.38.)

Let z denote the common parent of y and p , and note that z must be black, because y is red. The combination of y and z represents a 3-node in the corresponding $(2,4)$ tree T' . In this case, we perform a rotation about y and z , and then recolor y black and z red. This denotes a reorientation of a 3-node in the corresponding $(2,4)$ tree T' .

We now reconsider the double-black problem at p . After the adjustment, the sibling of p is black, and either Case 1 or Case 2 applies. Furthermore, the next application will be the last, because Case 1 is always terminal and Case 2 will be terminal given that the parent of p is now red.

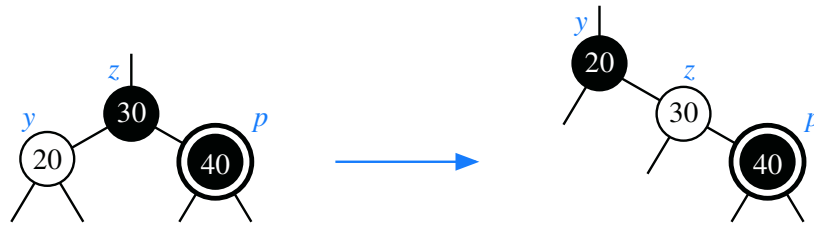


Figure 11.38: A rotation and recoloring about red node y and black node z in the presence of a double-black problem (a symmetric configuration is possible). This amounts to a change of orientation in the corresponding 3-node of a $(2,4)$ tree. This operation does not affect the black depth of any paths through this portion of the tree, but after the operation, one of the other resolutions to the double-black problem may be applied, as the sibling of p will be black.

In Figure 11.39, we show a sequence of deletions on a red-black tree. We illustrate a Case 1 restructuring in parts (c) and (d). We illustrate a Case 2 recoloring in parts (f) and (g). Finally, we show an example of a Case 3 rotation between parts (i) and (j), concluding with a Case 2 recoloring in part (k).

Performance of Red-Black Trees

The asymptotic performance of a red-black tree is identical to that of an AVL tree or a $(2,4)$ tree in terms of the sorted map ADT, with guaranteed logarithmic time bounds for most operations. (See Table 11.2 for a summary of the AVL performance.) The primary advantage of a red-black tree is that an insertion or deletion requires only a **constant number of restructuring operations**. (This is in contrast to AVL trees and $(2,4)$ trees, both of which require a logarithmic number of structural changes per map operation in the worst case.) That is, an insertion or deletion in a red-black tree requires logarithmic time for a search, and may require a logarithmic number of recoloring operations that cascade upward. We formalize these facts with the following propositions.

Proposition 11.9: *The insertion of an entry in a red-black tree storing n entries can be done in $O(\log n)$ time and requires $O(\log n)$ recolorings and at most one trinode restructuring.*

Proposition 11.10: *The algorithm for deleting an entry from a red-black tree with n entries takes $O(\log n)$ time and performs $O(\log n)$ recolorings and at most two restructuring operations.*

The proofs of these propositions are left as Exercises R-11.26 and R-11.27.

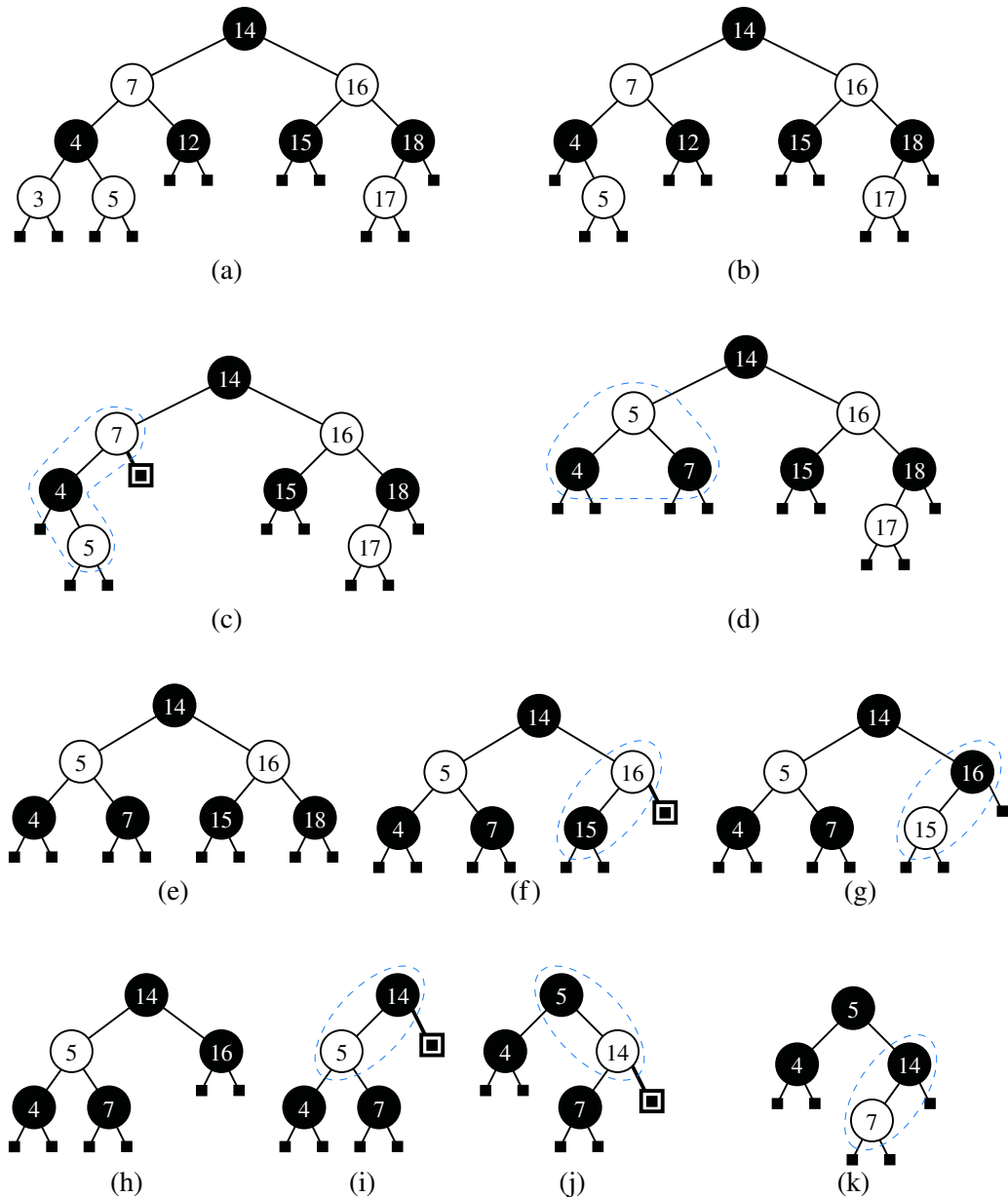


Figure 11.39: A sequence of deletions from a red-black tree: (a) initial tree; (b) removal of 3; (c) removal of 12, causing a black deficit to the right of 7 (handled by restructuring); (d) after restructuring; (e) removal of 17; (f) removal of 18, causing a black deficit to the right of 16 (handled by recoloring); (g) after recoloring; (h) removal of 15; (i) removal of 16, causing a black deficit to the right of 14 (handled initially by a rotation); (j) after the rotation the black deficit needs to be handled by a recoloring; (k) after the recoloring.

11.6.2 Java Implementation

In this section, we will provide an implementation of a `RBTreeMap` class that inherits from the standard `TreeMap` class and relies on the balancing framework described in Section 11.2.1. In that framework, each node stores an auxiliary integer that can be used for maintaining balance information. For a red-black tree, we use that integer to represent color, choosing to let value 0 (the default) designate the color black, and value 1 the color red; with this convention, any newly created leaf in the tree will be black.

Our implementation begins in Code Fragment 11.14, with constructors for an empty map, and a series of convenient utilities for managing the auxiliary field to represent color information. That code fragment continues with support for rebalancing the tree after an insertion is performed. When an entry has been inserted in a tree by the standard search-tree algorithm, it will be stored at a previously external node that was converted to an internal node with two new external children. The `rebalanceInsert` hook is then called, allowing us the opportunity to modify the tree. Except for the special case where the new element is at the root, we change the color of the node with the new element to red (it had been black when a leaf), and then we consider the possibility that we have a double-red violation. The `resolveRed` utility closely follows the case analysis described in Section 11.6.1, recurring in the case when the red violation is propagated upward.

Code Fragment 11.15 manages the rebalancing process after a deletion, based upon the case analysis described in Section 11.6.1. If the removed node was red, then no other action is necessary; however, if the removed node was black, we must consider a way to restore the depth property. An additional challenge is that by the time the `rebalanceDelete` method is called, a node has already been removed from the tree (this hook is invoked on the promoted *child* of that removed node). Fortunately, we can infer the properties of the removed node based upon the red-black tree properties, which were satisfied before the deletion.

In particular, let p denote the promoted child of the removed node. If a black node with a red child has been deleted, then p will be that red child; we remedy this by coloring p black. Otherwise, if p is not the root, let s denote the removed node's sibling (which will appear as p 's sibling after the deletion). If the deleted node was black with two black children, we must treat p as a **double black** node to be remedied. This is the case if, and only if, its sibling's subtree has a black internal node (because the red-black depth property was satisfied prior to the deletion). We therefore test whether s is a black internal node, or a red internal node with an internal node as a child (which must be black due to the red property of the tree).

We are able to detect the double-black problem within the `rebalanceDelete` method of Code Fragment 11.15, and we rely on the recursive `remedyDoubleBlack` method of that code fragment to resolve the problem.

```

1  /** An implementation of a sorted map using a red-black tree. */
2  public class RBTreeMap<K,V> extends TreeMap<K,V> {
3      /** Constructs an empty map using the natural ordering of keys. */
4      public RBTreeMap() { super(); }
5      /** Constructs an empty map using the given comparator to order keys. */
6      public RBTreeMap(Comparator<K> comp) { super(comp); }
7      // we use the inherited aux field with convention that 0=black and 1=red
8      // (note that new leaves will be black by default, as aux=0)
9      private boolean isBlack(Position<Entry<K,V>> p) { return tree.getAux(p)==0; }
10     private boolean isRed(Position<Entry<K,V>> p) { return tree.getAux(p)==1; }
11     private void makeBlack(Position<Entry<K,V>> p) { tree.setAux(p, 0); }
12     private void makeRed(Position<Entry<K,V>> p) { tree.setAux(p, 1); }
13     private void setColor(Position<Entry<K,V>> p, boolean toRed) {
14         tree.setAux(p, toRed ? 1 : 0);
15     }
16     /** Overrides the TreeMap rebalancing hook that is called after an insertion. */
17     protected void rebalanceInsert(Position<Entry<K,V>> p) {
18         if (!isRoot(p)) {
19             makeRed(p); // the new internal node is initially colored red
20             resolveRed(p); // but this may cause a double-red problem
21         }
22     }
23     /** Remedies potential double-red violation above red position p. */
24     private void resolveRed(Position<Entry<K,V>> p) {
25         Position<Entry<K,V>> parent,uncle,middle,grand; // used in case analysis
26         parent = parent(p);
27         if (isRed(parent)) { // double-red problem exists
28             uncle = sibling(parent);
29             if (isBlack(uncle)) { // Case 1: misshapen 4-node
30                 middle = restructure(p); // do trinode restructuring
31                 makeBlack(middle);
32                 makeRed(left(middle));
33                 makeRed(right(middle));
34             } else { // Case 2: overfull 5-node
35                 makeBlack(parent); // perform recoloring
36                 makeBlack(uncle);
37                 grand = parent(parent);
38                 if (!isRoot(grand)) {
39                     makeRed(grand); // grandparent becomes red
40                     resolveRed(grand); // recur at red grandparent
41                 }
42             }
43         }
44     }

```

Code Fragment 11.14: The RBTreeMap class. (Continues in Code Fragment 11.15.)

```

45  /** Overrides the TreeMap rebalancing hook that is called after a deletion. */
46  protected void rebalanceDelete(Position<Entry<K,V>> p) {
47      if (isRed(p)) // deleted parent was black
48          makeBlack(p); // so this restores black depth
49      else if (!isRoot(p)) {
50          Position<Entry<K,V>> sib = sibling(p);
51          if (isInternal(sib) && (isBlack(sib) || isInternal(left(sib))))
52              remedyDoubleBlack(p); // sib's subtree has nonzero black height
53      }
54  }
55
56  /** Remedies a presumed double-black violation at the given (nonroot) position. */
57  private void remedyDoubleBlack(Position<Entry<K,V>> p) {
58      Position<Entry<K,V>> z = parent(p);
59      Position<Entry<K,V>> y = sibling(p);
60      if (isBlack(y)) {
61          if (isRed(left(y)) || isRed(right(y))) { // Case 1: trinode restructuring
62              Position<Entry<K,V>> x = (isRed(left(y)) ? left(y) : right(y));
63              Position<Entry<K,V>> middle = restructure(x);
64              setColor(middle, isRed(z)); // root of restructured subtree gets z's old color
65              makeBlack(left(middle));
66              makeBlack(right(middle));
67          } else { // Case 2: recoloring
68              makeRed(y);
69              if (isRed(z))
70                  makeBlack(z); // problem is resolved
71              else if (!isRoot(z))
72                  remedyDoubleBlack(z); // propagate the problem
73          }
74      } else { // Case 3: reorient 3-node
75          rotate(y);
76          makeBlack(y);
77          makeRed(z);
78          remedyDoubleBlack(p); // restart the process at p
79      }
80  }
81  }

```

Code Fragment 11.15: Support for deletion in the RBTreeMap class (continued from Code Fragment 11.14).